

Müllabfalltrennung

Computer Vision und Deep Learning zur Erkennung von Tetrapak und Dosen



KI-generiertes Symbolbild zur Visualisierung der Projektidee.

Autoren: Anna Skrylnyk
Emrah Tekin

Kurs: Einführung AI

Dozent: Gion Sialm

E-Mail: Gion.Sialm@fhgr.ch

Institution: Fachhochschule Graubünden

Datum: 11. Mai 2026

GitHub Repository:

https://github.com/EmrahTek/YOLO_waste_detection/tree/main

Inhaltsverzeichnis

1	Einleitung	2
2	Projektidee	2
3	Projektablauf	3
4	Hardware, Beleuchtung und Konstruktion	4
5	Warum Deep Learning und warum nicht klassische Bildverarbeitung?	5
6	Software	6
7	Interpretation	7
8	Fazit	8
9	Reflexion	9
10	Quellen	10

1 Einleitung

Im Rahmen des Kurses Einführung AI sollte ein Projekt mit Computer Vision und Deep Learning umgesetzt werden. Für die Wahl eines passenden Themas im Bereich Computer Vision gab es mehrere Gespräche in der Gruppe. Danach wurde entschieden, an einem Thema im Bereich Umwelt und Recycling zu arbeiten.

In diesem Zusammenhang wurde das Computer-Vision-Projekt Müllabfalltrennung von der Data-Science-Studentin Anna Skrylnyk und dem Photonik-Studenten Emrah Tekin auf Demo-Niveau entwickelt.

2 Projektidee

Wenn man das touristische Potenzial des Kantons Graubünden betrachtet, kann es für Touristinnen und Touristen oft schwierig sein, nach ihrem Besuch den Müll richtig zu entsorgen. Um dieses Problem zu unterstützen, sollte eine Lösung entwickelt werden, bei der grosse Müllcontainer mit integrierter Computer Vision ausgestattet werden.

Dabei sollen die Personen beim Trennen ihres Mülls unterstützt werden. Da auch die Kosten berücksichtigt wurden, wurde entschieden, ein relativ günstiges System zu entwickeln. Deshalb wurde für das Demo-Projekt ein Raspberry Pi 5 mit einer Sony AI Pi Camera ausgewählt [6, 7, 8].

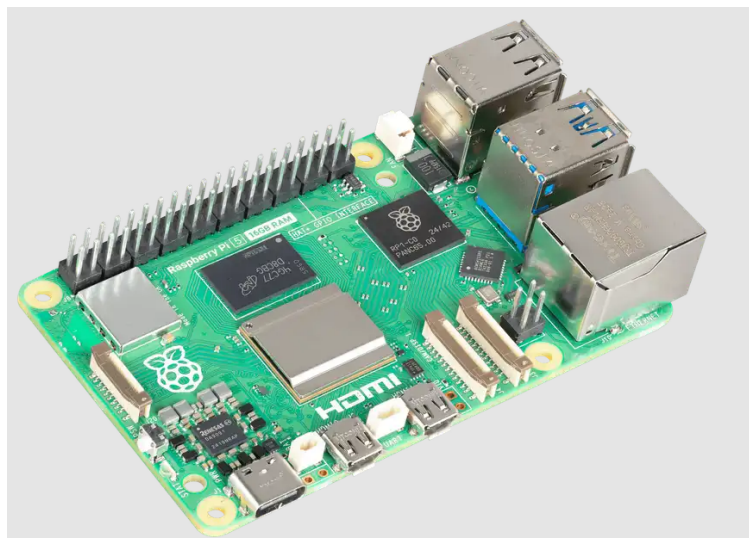


Abbildung 1: Raspberry Pi 5 als Hardware-Plattform für das Demo-Projekt.



Abbildung 2: Raspberry Pi AI Camera für die spätere Bilderfassung.

3 Projektablauf

Da es sich um ein Deep-Learning-Projekt handelt, musste zuerst ein Datensatz erstellt werden. Am Anfang wurde versucht, sowohl fertige Datensätze aus dem Internet als auch Bilder aus dem sozialen Umfeld zu sammeln.

Da das Projekt von zwei Personen in begrenzter Zeit fertiggestellt werden sollte, wurde es auf zwei Klassen beschränkt: Tetrapak und Dose. Insgesamt wurden 657 Fotos gesammelt. Laut der Ultralytics-Dokumentation sollte ein realistisches Modell mindestens ungefähr 1500 Fotos haben [1]. Trotzdem wurde das Projekt auf zwei Klassen begrenzt.

Nachdem der Datensatz erstellt wurde, wurde er in Train 63 %, Validation 22 % und Test 14 % aufgeteilt. Dabei traten auch die Schwierigkeiten eines kleinen Datensatzes auf.

Tabelle 1: Aufteilung des Datensatzes

Split	Bilder	Label-Dateien	Objekte
Train	247	247	431
Validation	87	87	175
Test	57	57	65
Gesamt	391	391	671

Bei einem kleinen Datensatz kann das Risiko für Overfitting steigen. Das Modell kann echte Variationen nicht ausreichend sehen. Auch die Metriken können weniger zuverlässig sein. Wegen der begrenzten Zeit wurde das Projekt trotzdem mit diesem Datensatz fortgesetzt.

Danach wurde der Datensatz mit der Anwendung CVAT annotiert. Anschließend wurde er im Ultralytics-YOLO-Format exportiert [9]. Wir hätten auch fertige Datensätze verwenden können. Aber

wir wollten die einzelnen Schritte eines Computer-Vision-Projekts besser verstehen. Deshalb haben wir versucht, das Projekt von Anfang an selbst aufzubauen.

Nach dem Export der CVAT-Annotationen wurde mit Hilfe von ChatGPT die Softwarestruktur des Projekts erstellt. Die Kritik am Softwareteil wird in einem späteren Abschnitt des Berichts erklärt.

Das Modell wurde zuerst auf einem Laptop mit einer NVIDIA 2000 Ada Grafikkarte trainiert, validiert und getestet. Die Ergebnisse wurden bewertet. Danach wurde das Projekt auf GitHub hochgeladen. Für die spätere Übertragung auf den Raspberry Pi 5 wurde ein NCNN-Modell erstellt [5, 11]. Danach wurde das Projekt auf den Raspberry Pi 5 übertragen.

Später wurde mit Fusion 360 eine passende Plattform entwickelt.

4 Hardware, Beleuchtung und Konstruktion

Als Hardware wurde ein Raspberry Pi 5 verwendet, weil er im Vergleich zu einem Computer relativ günstig ist, weniger Energie verbraucht und für Edge-Detection-Anwendungen geeignet ist [6].

Für den Raspberry Pi 5 wurde keine zusätzliche GPU-Karte verwendet. Stattdessen wurde die in der Sony AI Camera integrierte GPU genutzt [7, 8].

Für die Beleuchtung wurde ein diffuses Ringlicht verwendet. Der Grund dafür ist, dass eine homogene Beleuchtung bei Computer-Vision-Anwendungen bessere Ergebnisse liefern kann.

Um diese Komponenten stabil, fest und gegen Vibrationen geschützt zusammenzubringen, wurde für die Demo ein Gehäuse entwickelt. In diesem Zusammenhang wurde mit Fusion 360 ein Design erstellt und anschließend mit einem 3D-Drucker aus PLA-Filament gedruckt.

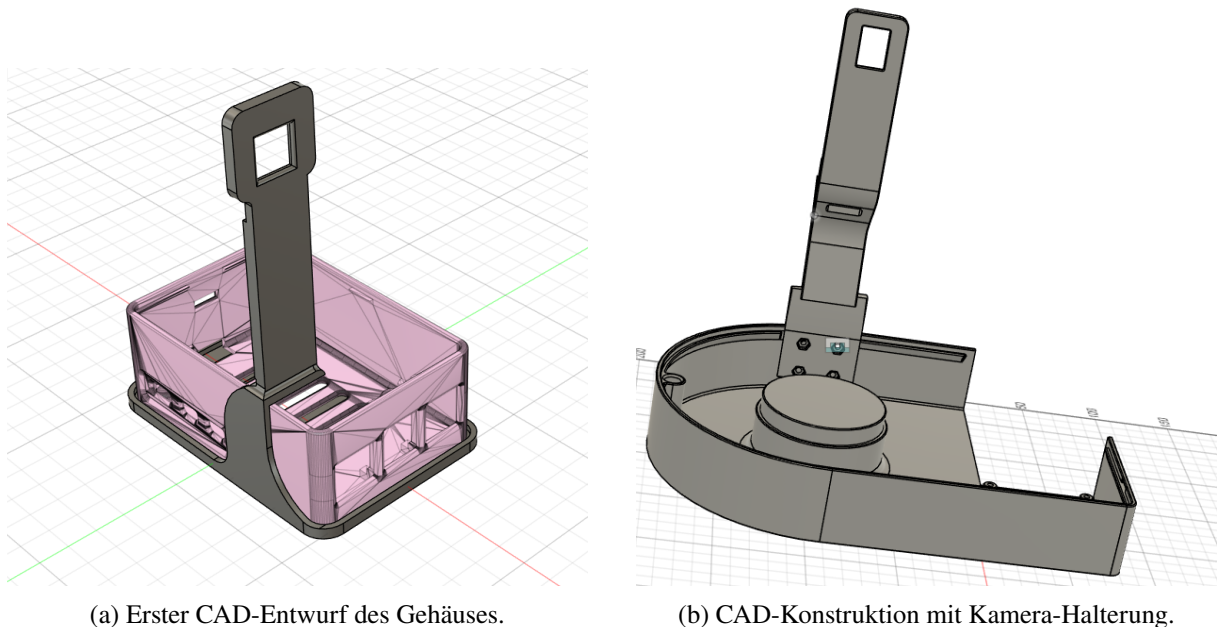
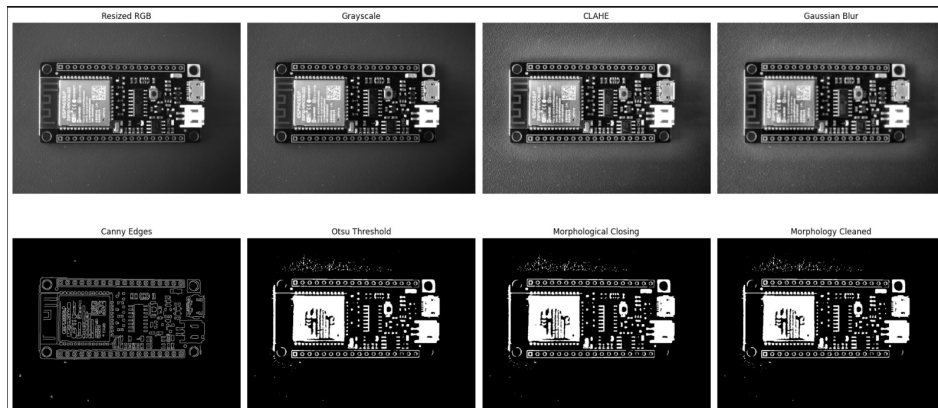


Abbildung 3: Fusion-360-Konstruktion der Plattform und Kamera-Halterung.

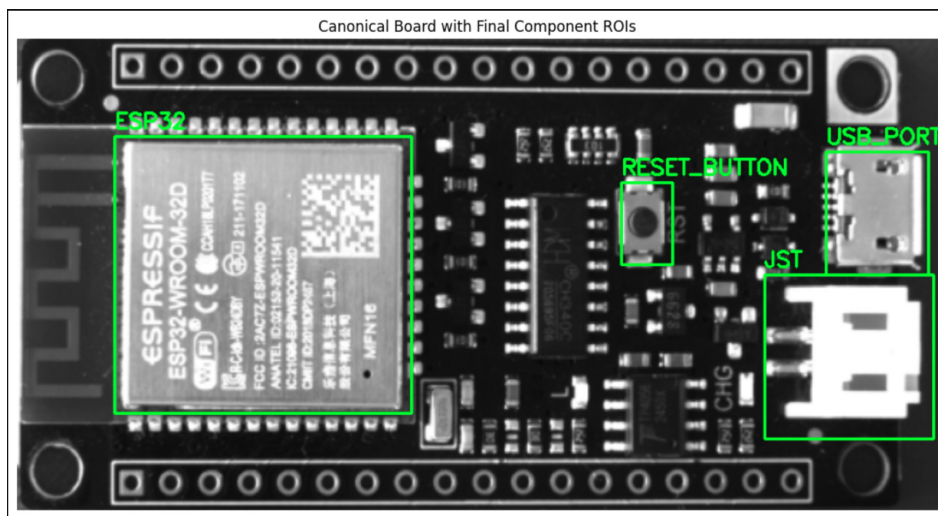
5 Warum Deep Learning und warum nicht klassische Bildverarbeitung?

Mit klassischer Bildverarbeitung kann man mit weniger Daten, einer festen Kamera und einer kontrollierten Umgebung relativ schnell eine Bildverarbeitung durchführen.

Zum Beispiel wurde im Kurs Bildverarbeitung für die PCB-Bauteilerkennung nur für eine bestimmte Platine auf einer festen Plattform mit Filtern gearbeitet. Wenn dem Programm aber eine andere Platine gezeigt worden wäre, hätte es keine zuverlässige Vorhersage machen können.



(a) Beispiel für klassische Bildverarbeitungsschritte wie Graustufenbild, CLAHE, Canny Edges und Morphologie.



(b) Beispiel für feste ROIs bei einer PCB-Bauteilerkennung.

Abbildung 4: Beispiel aus klassischer Bildverarbeitung: feste Bildverarbeitungsschritte und feste Komponentenbereiche.

Für dieses Projekt war klassische Bildverarbeitung nicht gut geeignet. Deshalb musste ein Modell mit einem Convolutional Neural Network trainiert werden.

Wenn sich Szene, Produkt, Farbe oder Beschriftung ändern, kann ein trainiertes Modell bessere Ergebnisse liefern. Die Schwierigkeit bei CNNs war jedoch, einen grossen Datensatz zu bekommen.

Bei YOLO schreiben wir die Regeln nicht wie bei klassischer Bildverarbeitung selbst. Das Modell lernt aus Beispielen [1].

6 Software

Bevor das Programm geschrieben wurde, wurde zuerst die Entwicklungsumgebung vorbereitet. Für die Versionskontrolle wurde im Team ein Repository auf GitHub erstellt und dort weitergeführt [12].

Als Programmiersprache wurde Python verwendet. Dafür wurde mit `venv` eine virtuelle Umgebung erstellt. Die für das Projekt notwendigen Pakete wurden in der Datei `requirements.txt` angegeben.

Zu den wichtigsten Paketen im Projekt gehören Ultralytics, OpenCV und NCNN [1, 10, 11]. OpenCV wurde besonders für Bildverarbeitung genutzt [10]. Ultralytics wurde für die YOLO-Modelle verwendet [1]. NCNN wurde für das Nano-Modell auf dem Raspberry Pi genutzt [11].

In der Softwarephase haben wir zunächst Vibe Coding mit dem Codex-Agenten getestet. Da der Dozent jedoch mehrfach darauf hingewiesen hat, Vibe Coding nicht zu verwenden und stattdessen GenAI-Tools als Assistenzsysteme einzusetzen, haben wir diesen Ansatz verworfen. Entsprechend den Kursvorgaben nutzten wir ChatGPT als unterstützendes Werkzeug und überprüften alle generierten Antworten kritisch.

Aufgrund der Inhalte aus den GenAI-Lektionen und der Empfehlungen des Dozenten haben wir Vibe Coding aufgegeben und ChatGPT als Assistenten verwendet. Wir haben die Antworten von ChatGPT kritisch hinterfragt.

Zum Beispiel hatte ChatGPT beim Training des Modells 100 Epochen vorgeschlagen. Wir haben gefragt, warum diese Zahl gewählt wurde. ChatGPT hatte uns auch vorgeschlagen, `yolo11n.pt` zu verwenden. Danach haben wir aber zusätzlich auch `yolo10.pt` und `yolo26.pt` getestet [2, 3, 4].

Beim Vergleich der drei Modelle wurden Tabellen und Diagramme mit Matplotlib erstellt. Trotz des kleinen Datensatzes war das Modell auf Demo-Niveau nicht schlecht. Für bessere Ergebnisse wären aber mehr Datensätze aus unterschiedlichen Szenen notwendig.

Wir haben versucht, ChatGPT als guten Assistenten zu verwenden. Wir haben Fragen gestellt wie: Warum ist das so? Was würde passieren, wenn wir es anders machen? Wie könnten wir den Recall-Wert des Modells verbessern? Warum war der Recall-Wert bei Tetrapak niedriger als bei Dose? Welche Wirkung hat das auf das Modell?

Wir haben mit ChatGPT fast wie in einer Diskussion gearbeitet. Die Aussagen von ChatGPT wurden mit Quellen verglichen. Wir haben die Links geöffnet und die Informationen dort kontrolliert.

Im Programmierteil haben wir vielleicht keinen sehr grossen Code selbst geschrieben. Aber wir haben einige Parameter geändert und getestet.

7 Interpretation

Bei der Bewertung des Modells haben wir auf Empfehlung des Dozenten die Bewertungsmetriken von der Ultralytics-Seite berücksichtigt [1]. Es ist sinnvoll, diese Metriken im Rahmen des Projekts kurz zu erklären.

Tabelle 2: Bewertungsmetriken im Rahmen des Projekts

Metrik	Definition im Rahmen des Projekts
Precision	Precision zeigt, ob die Objekte, die das Modell als Tetrapak oder Dose markiert, wirklich korrekt sind.
Recall	Recall zeigt, wie viele der tatsächlich vorhandenen Tetrapak- und Dosen-Objekte vom Modell gefunden wurden.
F1-Score	Der F1-Score zeigt das Gleichgewicht zwischen Precision und Recall. Er bewertet also zusammen, wie gut das Modell richtige Vorhersagen macht und wie wenige Objekte es übersieht.
mAP50	mAP50 misst, wie gut das Modell Objekte mit der richtigen Klasse und ungefähr der richtigen Position bei einer IoU-Grenze von 0.50 erkennt.
mAP50-95	mAP50-95 ist eine strengere Metrik. Sie bewertet, wie genau das Modell die Positionen der Objekte bei verschiedenen IoU-Grenzen erkennt.

Zuerst wollten wir das `best.pt`-Modell aus dem Projekt mit diesen Metriken bewerten. Dieses Modell basierte auf `yolo11n.pt` [3].

YOLO11 wurde uns von ChatGPT vorgeschlagen. Wir haben gefragt, warum dieses Modell gewählt wurde. ChatGPT erklärte, dass es häufig verwendet wird.

Ausserdem sagte ChatGPT, dass das aktuelle Modell YOLO26 bessere Ergebnisse liefern könnte. Deshalb haben wir das Modell auch mit YOLO26 und zusätzlich mit YOLO10 erneut trainiert und getestet [2, 4].

Nachdem wir Vergleichstabellen und Diagramme erstellt hatten, entschieden wir, dass `yolo11n.pt` das beste Ergebnis geliefert hat. Die Einschätzung von ChatGPT zu YOLO26 war in diesem Fall nicht richtig.

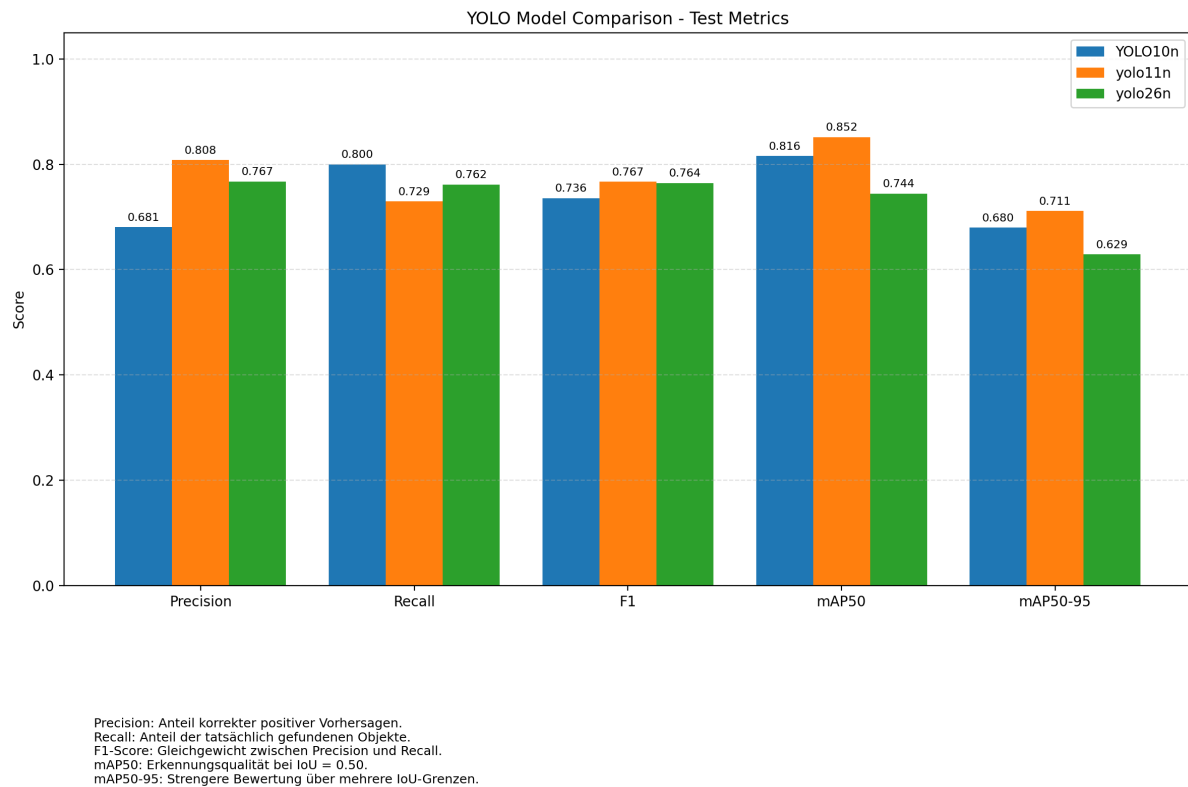


Abbildung 5: Vergleich der Modelle YOLO10n, YOLO11n und YOLO26n mit Testmetriken.

Tabelle 3: Vergleich von YOLO10n, YOLO11n und YOLO26n

Metrik	YOLO10n	YOLO11n	YOLO26n	Bestes Modell
Precision	0.6809	0.8080	0.7672	YOLO11n
Recall	0.8000	0.7292	0.7617	YOLO10n
F1	0.7356	0.7666	0.7645	YOLO11n
mAP50	0.8158	0.8516	0.7442	YOLO11n
mAP50-95	0.6798	0.7113	0.6286	YOLO11n

8 Fazit

Das Projekt Müllabfalltrennung hat auf Demo-Niveau das gewünschte Ziel erreicht. Die Projektidee unterstützt ein Umweltproblem im Kanton, in dem wir leben, und passt auch zum Thema Tourismus und Nachhaltigkeit.

Das Projekt wurde ganzheitlich betrachtet. Es wurde nicht nur der Softwareteil mit Hilfe von künstlicher Intelligenz erstellt und danach beendet. Auch Budget, Mobilität und Energieverbrauch wurden berücksichtigt. Dafür wurden die notwendigen Hardware-Vorbereitungen gemacht.

Nach den Hardware- und Elektronikvorbereitungen wurde eine erste Prototyp-Plattform als CAD-Konstruktion entwickelt. Ziel war es, die Komponenten stabiler, widerstandsfähiger gegen Vibrationen und langlebiger zu machen.

Im Projekt wurde nicht nur ein fertiger Datensatz verwendet. Um auch in unserem sozialen Umfeld

Aufmerksamkeit für das Thema zu schaffen, wurden eigene Fotos gesammelt. Diese Bilder wurden anschließend mit CVAT annotiert.

In der Softwarephase wurde mit ChatGPT eine Art gegenseitige Diskussion geführt. Verschiedene Modelle wurden getestet und die Ergebnisse wurden mit Tabellen und Diagrammen visualisiert.

Während des Projekts wurden Quellen und Links verwendet. Generative AI wurde als Assistent genutzt. Mit Deep Learning wurde ein Modell entwickelt, das einen kleinen Beitrag zum Umweltschutz leisten kann. Damit wurde ein Demo-Projekt auf das gewünschte Niveau gebracht.

9 Reflexion

Während des Projekts haben wir gesehen, dass ein Deep-Learning-Projekt nicht nur aus dem Training eines Modells besteht. Zuerst mussten Bilder gesammelt, annotiert und in ein passendes YOLO-Format gebracht werden. Danach wurden verschiedene Modelle trainiert, validiert und miteinander verglichen. Dadurch konnten wir den ganzen Ablauf eines Computer-Vision-Projekts besser verstehen.

Eine wichtige Schwierigkeit war der kleine Datensatz. Obwohl das Modell auf Demo-Niveau funktioniert hat, sind die Ergebnisse noch nicht vollständig allgemein gültig. Das Modell hat nur eine begrenzte Anzahl von Beispielen gesehen. Deshalb kann es bei anderen Lichtverhältnissen, Hintergründen oder Kameraperspektiven schlechter funktionieren. Für bessere Ergebnisse wären mehr Bilder aus realen Situationen notwendig.

Auch die Arbeit mit Generative AI war ein wichtiger Teil des Projekts. Am Anfang wurde Vibe Coding mit einem Codex Agent ausprobiert. Dabei haben wir aber gemerkt, dass man schnell die Kontrolle über den Code verlieren kann. Danach haben wir ChatGPT eher als Assistenten verwendet. Wir haben die Vorschläge nicht einfach übernommen, sondern immer wieder nachgefragt und die Antworten kritisch geprüft.

Besonders interessant war der Vergleich zwischen YOLO10n, YOLO11n und YOLO26n. Am Anfang war die Erwartung, dass ein neueres Modell automatisch bessere Ergebnisse liefert. Die Tests haben aber gezeigt, dass YOLO11n in unserem Datensatz die beste Gesamtleistung erreicht hat. Daraus haben wir gelernt, dass die Modellwahl immer vom Datensatz, vom Zielsystem und von den Anforderungen abhängt.

Auch die Hardware war ein wichtiger Lernpunkt. Auf dem Laptop ist das Training und Testen einfacher und schneller. Der Raspberry Pi 5 ist dagegen weniger leistungsstark, aber günstiger, mobiler und besser für ein Edge-AI-Demo geeignet. Deshalb mussten nicht nur die Modellmetriken, sondern auch Energieverbrauch, Mobilität, Kosten und reale Anwendung berücksichtigt werden.

Für die Zukunft wäre es sinnvoll, mehr Daten zu sammeln, besonders mit der Raspberry-Pi-Kamera. Außerdem könnten mehr Lichtbedingungen, verschiedene Hintergründe und teilweise verdeckte Objekte aufgenommen werden. Dadurch könnte das Modell robuster werden und besser in realen Situationen funktionieren.

10 Quellen

Literatur

- [1] Ultralytics. *Ultralytics YOLO Documentation*. Online verfügbar unter: <https://docs.ultralytics.com/>. Zugriff am: 12. Mai 2026.
- [2] Ultralytics. *YOLOv10 Documentation*. Online verfügbar unter: <https://docs.ultralytics.com/models/yolov10/>. Zugriff am: 12. Mai 2026.
- [3] Ultralytics. *YOLO11 Documentation*. Online verfügbar unter: <https://docs.ultralytics.com/models/yolo11/>. Zugriff am: 12. Mai 2026.
- [4] Ultralytics. *YOLO26 Documentation*. Online verfügbar unter: <https://docs.ultralytics.com/models/yolo26/>. Zugriff am: 12. Mai 2026.
- [5] Ultralytics. *Model Export with Ultralytics YOLO*. Online verfügbar unter: <https://docs.ultralytics.com/modes/export/>. Zugriff am: 12. Mai 2026.
- [6] Raspberry Pi Ltd. *Raspberry Pi 5*. Online verfügbar unter: <https://www.raspberrypi.com/products/raspberry-pi-5/>. Zugriff am: 12. Mai 2026.
- [7] Raspberry Pi Ltd. *Raspberry Pi AI Camera*. Online verfügbar unter: <https://www.raspberrypi.com/products/ai-camera/>. Zugriff am: 12. Mai 2026.
- [8] Sony Semiconductor Solutions. *Raspberry Pi AI Camera*. Online verfügbar unter: <https://www.ai-trios.sony-semicon.com/edge-ai-devices/raspberry-pi-ai-camera>. Zugriff am: 12. Mai 2026.
- [9] CVAT. *Ultralytics YOLO Format*. Online verfügbar unter: https://docs.cvat.ai/docs/dataset_management/formats/format-yolo-ultralytics/. Zugriff am: 12. Mai 2026.
- [10] OpenCV. *OpenCV Documentation*. Online verfügbar unter: <https://docs.opencv.org/>. Zugriff am: 12. Mai 2026.
- [11] Tencent. *NCNN GitHub Repository*. Online verfügbar unter: <https://github.com/Tencent/ncnn>. Zugriff am: 12. Mai 2026.
- [12] Tekin, E.; Skrylnyk, A. *YOLO Waste Detection GitHub Repository*. Online verfügbar unter: https://github.com/EmrahTek/YOLO_waste_detection/tree/main. Zugriff am: 12. Mai 2026.